Graphs

Table Of Contents

What is a graph

Graphs are a collection of vertices and edges. We have a few different kind of graphs.

Each vertex within a graph is able to connect to other vertices via edges.

Edges have two points, a start and end which refer to vertices. These edges could loop to the same vertex.

- Directed Graphs
- Undirected Graphs
- Weighted Graphs
- Unweighted Graphs (or weights of 1)

What is the significance with graphs?

As outlined with the scenarios prior, even seemingly simple applications are going to involve graphs. This is because what we end up modelling usually has indicates that entities have relations and each entity type is going to interact with another in some form.

Graph definitions

- Vertex, usually also called node
- Edge, which are either a vertex pair or contains more data that describes the connection.

Some scenarions where graphs are used.

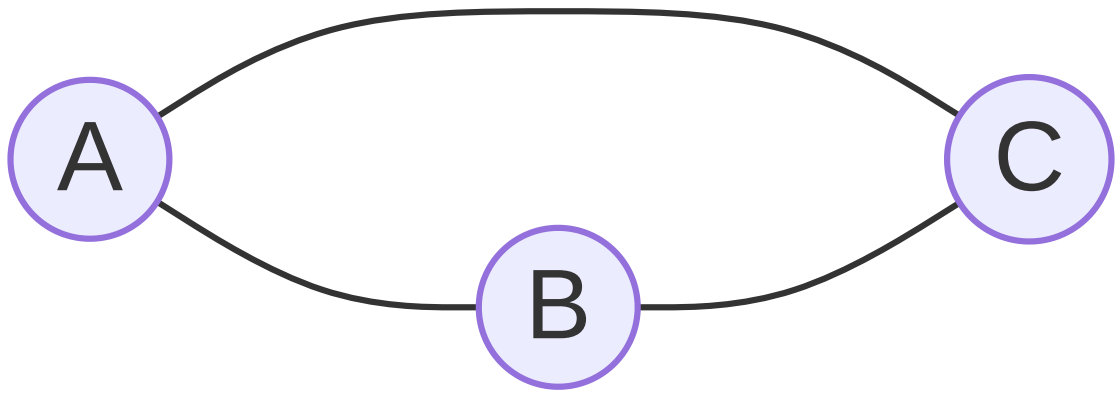
- Airport Flights and connections
- Google maps, routing from one location to another
- A state machine where we want to know what state we are in and where we transition town

Unweighted and Undirected Graph

A common graph is a graph where the edges do not have a weight (usually implying there is a cost associated with moving a direction) and a direction, when undirected it means that going from one node to another can also be reversed.

Undirected Graph

Below we have an undirected graph, in this form, the edges are considered unweighted as well.



Directed and Weighted Graph

When describing an edge, an edge will have a cost associated.

A common scenario would be "Finding out how much fuel would be used by taking a set of nodes/path"

Or another "How much would I pay if I was to take these flights".

We can then compare paths and potentially find out what is better value.

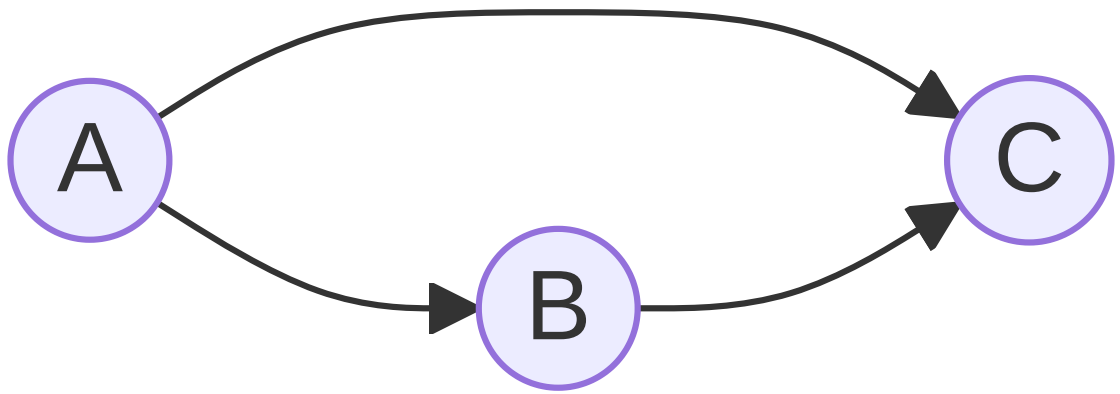
Can we have Weighted and Undirected? Yes

Can we have Unweighted and Directed? Yes

Just wanted to clarify those points.

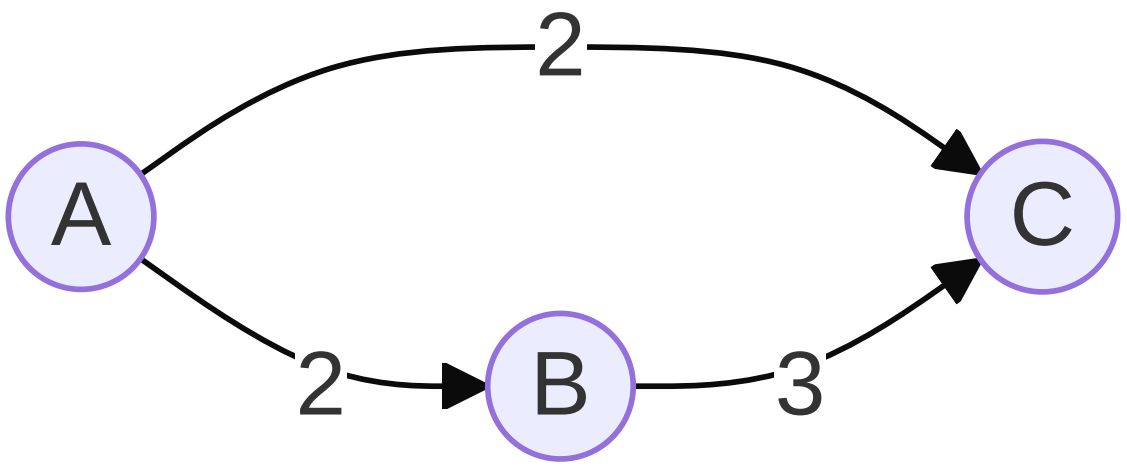
Directed Graph

Within this case, we have a directed graph that we can identify with the arrows being formed to indicate the direction of the edge.



Directed and Weighted Graph

Weights are assigned to the edges, it is used to indicate the cost of going from one node to another. In the example below, we have a weights associated with the edges.



Traversals

Similar to trees, graphs also have a method to traverse through the graph.

- BFS - Breadth First Search
- DFS - Depth First Search

The above two are used for discovering all nodes within the graph.

Why are they important?

For graphs and trees, these two algorithms establish some important patterns and foundations.

- BFS is used for identitfying reachability of nodes, evaluating options and paths and identifying layers. It can be used to establish a tree from a graph from a selected node.
- DFS is used for identifying solutions (or geedily looking for a solution) before backtracking, commonly used in web crawlers and maze solvers.

BFS

The below is the BFS algorithm.

```
BFS(u) {
    Q = [u]
    while !Q.empty() {
        cur = Q.dequeue()

        foreach(n in cur.neighbours) {
            if n is unvisited {
                Q.enqueue(v);
            }
        }
    }
}
```

The main perk with a BFS is that will visit all nodes in an order that they have been detected, typically forming a tree.

DFS

An alternative algorithm, is using a depth-first search. We switch out a queue for a stack.

This is because the next node we retrieve gets it from the top of the stack.

```
DFS(u) {
    S = [u]
    while !S.empty() {
        cur = Q.pop()

        foreach(n in cur.neighbours) {
            if n is unvisited {
                Q.push(v);
            }
        }
    }
}
```

Visit: https://visualgo.net/en/dfsbfs

- Step through the BFS algorithm and observe how the algorithm is executed
- Step through the DFS algorithm and observe how the algorithm is executed

Shortest Path

The main algorithm we are focusing on is "Dijkstra's Shortest path".

This algorithm changes the selection process of a BFS algorithm to get the next-minimum cost node visited.

```
ShortestPath(u) {
    PQ = [u]
    while !PQ.empty() {
        [root, cur] = Q.dequeueMin()

        foreach(n in cur.neighbours) {
            if n is unvisited {
                //Associate with cur,
                //So we know the path
                Q.enqueue((cur, n));
            }
        }
    }
}
```

Shortest Path

What is the advantage of this? The advantage of a shortest path algorithm is that allows us to find paths efficiently rather than having to find the reachability from one point and compare against them all.

It is also an efficient greedy algorithm.

Visit: https://visualgo.net/en/sssp

- Step through Dijkstra and observe how the algorithm is evaluated, you may need to construct an interesting graph to make a little more sense of it.

Exercises

1. Connecting and building rooms. The main class you need to build is called `Room`.
   A room will consist of:
   - A name which is a string
   - 4 connections `North`, `East`, `South`, `West`
   - Each connection leads to a different rooms

You will need to also build a `Visitor`, a `Visitor` has a
* A name which is a string
* `Room` they are currently in

Also create input handling where the user can specify `North`, `East`, `South` and `West` and change the `Visitor`'s room.

Resources

1. BFS & DFS, Visualgo, https://visualgo.net/en/dfsbfs
2. Dijkstra's Algorithm, Wikipedia, https://en.wikipedia.org/wiki/Dijkstra's_algorithm
3. Graph Data Structures, Visualgo, https://visualgo.net/en/graphds
4. Iterator Pattern, Wikipedia, https://en.wikipedia.org/wiki/Iterator_pattern